

Consumer Groups, Offsets & Transactions

A hands-on learning guide built from real questions and experiments — covering Consumer Groups, Partition Rebalancing, Offset Management, and Exactly-Once Transactions with working Python code.

Ch 1	Consumer Groups & Partition Rebalancing
Ch 2	Partition Key — Why All Messages Went to Partition 0
Ch 3	Offset Management — All Commit Types Explained
Ch 4	Auto-Commit Deep Dive — Three Real Scenarios
Ch 5	Independent Consumer Groups — How Offsets Are Isolated
Ch 6	Exactly-Once Transactions — Full Code Walkthrough

Chapter 1

Consumer Groups & Partition Rebalancing

What they are, why they exist, and how Kafka redistributes work automatically

The Intuition — The Pizza Restaurant

Imagine you run a pizza restaurant and 100 orders arrive on a conveyor belt. You have three chefs. You naturally divide the belt into three sections — chef A takes orders 1–33, chef B takes 34–66, chef C takes 67–100. If chef B calls in sick, you redistribute the belt between A and C. That is **exactly** how Kafka consumer groups work.

A **Consumer Group** is a set of consumers that cooperate to consume a topic. Kafka assigns each partition to exactly one consumer in the group at any moment. This gives you parallel processing AND guarantees ordering within a partition.

Why Are Consumer Groups Needed?

- One consumer cannot keep up if millions of messages pour in per second.
- A group splits the load — Consumer A reads partitions 0 and 1, Consumer B reads 2 and 3.
- They work in parallel without stepping on each other's messages.
- The key: two consumers share the same **group.id** — that is what tells Kafka they are a team.

How to Create a Consumer Group

A consumer group is created simply by giving two or more consumers the same **group.id**. Kafka handles the rest — no explicit creation step is needed.

Python — creating a consumer group

```
from confluent_kafka import Consumer
# Consumer A — starts the group
consumer_a = Consumer({
    'bootstrap.servers': 'localhost:9092',
    'group.id': 'order-group', # <-- the group name
    'auto.offset.reset': 'earliest',
})
consumer_a.subscribe(['orders'])
# Consumer B — joins the SAME group (different terminal / process)
```

```

consumer_b = Consumer({
    'bootstrap.servers': 'localhost:9092',
    'group.id': 'order-group', # <-- same group.id
    'auto.offset.reset': 'earliest',
})
consumer_b.subscribe(['orders'])
# Kafka automatically splits partitions between A and B

```

The Golden Rule: Max Useful Consumers = Partition Count

If your topic has 4 partitions and you have 6 consumers in a group, only 4 consumers get work. The remaining 2 sit idle — they cost resources but process nothing.

Scenario	Partitions	Consumers	Result
Balanced	4	4	Each consumer → 1 partition ✓
Under-scaled	4	2	Each consumer → 2 partitions (more load)
Over-scaled	4	6	4 active + 2 idle consumers ✗

What Is Rebalancing and Why Is It Needed?

Kafka has no permanent boss assigning work. When any consumer joins or leaves, Kafka must re-divide the partitions among whoever is currently active. This automatic re-division is called a **rebalance**.

Rebalancing is triggered when:

- A new consumer joins the group
- A consumer crashes or is shut down
- A consumer stops sending heartbeats (too slow or hung)

■ The pause problem

During a rebalance ALL consumption stops — this is called a stop-the-world event.

For high-throughput systems even a 2–3 second pause can cause downstream latency spikes.

Kafka 2.4+ introduced cooperative rebalancing which only moves partitions that need to move — dramatically reducing downtime.

Observing Assignments and Rebalances in Code

Kafka calls your callback functions when it gives you or takes partitions away. This is how you observe rebalancing happening in real time.

Python — observing rebalance callbacks

```
def on_assign(consumer, partitions):
    # Called when Kafka gives us partitions after a rebalance
    nums = [p.partition for p in partitions]
    print(f'=== ASSIGNED partitions: {nums} ===')
    def on_revoke(consumer, partitions):
        # Called when Kafka is about to take partitions away
        nums = [p.partition for p in partitions]
        print(f'=== REVOKED partitions: {nums} ===')
        # Best practice: commit offsets before losing partitions
        consumer.commit(asynchronous=False)
    consumer = Consumer({
        'bootstrap.servers': 'localhost:9092',
        'group.id': 'order-group',
        'enable.auto.commit': 'false',
        'session.timeout.ms': '10000', # 10s – detect crash faster
        'heartbeat.interval.ms': '3000', # heartbeat every 3s
    })
    consumer.subscribe(['orders'], on_assign=on_assign, on_revoke=on_revoke)
    while True:
        msg = consumer.poll(1.0)
        if msg and not msg.error():
            print(f'Got: {msg.value()} on P{msg.partition()}')
```

Expected terminal output when a second consumer joins:

Terminal output

```
=== ASSIGNED partitions: [0, 1, 2, 3] === # started alone, got all 4
Got: b'Order 1' on P0
Got: b'Order 2' on P2
# --- second consumer joins ---
=== REVOKED partitions: [0, 1, 2, 3] === # rebalance starts
=== ASSIGNED partitions: [0, 1] === # kept half
# second consumer gets [2, 3]
```

Inspect Groups from the Terminal

CLI — inspecting a consumer group

```
# See all consumer groups
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --list

# Describe a group – see partitions, offsets, lag
kafka-consumer-groups.sh \
```

```
--bootstrap-server localhost:9092 \  
--group order-group \  
--describe  
  
# Output columns:  
# CONSUMER-ID TOPIC PART CURRENT-OFFSET LOG-END LAG  
# consumer-1-abc orders 0 5432 5432 0 <- caught up  
# consumer-1-abc orders 1 5410 5450 40 <- 40 behind!  
# consumer-2-def orders 2 5432 5432 0
```

✓ Best Practices

Set `session.timeout.ms` lower for faster failure detection — but not below $3 \times \text{heartbeat.interval.ms}$.

Use cooperative-sticky partitioner (`partition.assignment.strategy=cooperative-sticky`) in production.

Match your consumer count to your partition count for optimal throughput.

Always call `consumer.close()` on shutdown — triggers `on_revoke` and commits offsets cleanly.

Chapter 2

Partition Key — Why All Messages Went to Partition 0

Diagnosing a real output, understanding sticky partitioning, and the fix

The Real Output You Saw

After running your producer and two consumers, the output showed something unexpected — both consumers were reading from **partition 0 only**, and Consumer B stopped after a few messages. Here is the actual output pattern:

Your actual terminal output

```
# Producer output – ALL messages going to partition [0]
Message delivered to orders [0] at offset 886868
Message delivered to orders [0] at offset 886869
Message delivered to orders [0] at offset 886870
# ... 886,000+ messages – all on partition 0
# Consumer A – reading recent messages
Consumed by Consumer A: Order 851 from partition 0 at offset 861
Consumed by Consumer A: Order 852 from partition 0 at offset 862
# Consumer B – reading old messages, then stopped
Consumed by Consumer B: Order 73 from partition 0 at offset 83
Consumed by Consumer B: Order 74 from partition 0 at offset 84
# Consumer B stops at offset 94 and goes silent
```

Root Cause: Sticky Partitioning + Single Partition

Two problems combined to cause this:

- **No key on your messages** — when a producer sends without a key, Kafka uses 'sticky partitioning': it picks one partition and sends everything there until the batch is full.
- **Topic likely had only 1 partition** — with 1 partition and 2 consumers, only 1 consumer gets work. Consumer B read old messages briefly before the rebalance gave partition 0 to Consumer A, leaving B idle.

How Kafka Decides Which Partition to Use

Situation	Kafka decision	Result
-----------	----------------	--------

No key given	Sticky partitioning	All messages → same partition
Key given	hash(key) % num_partitions	Same key always → same partition
Partition forced	Your explicit choice	Goes exactly where you say

The Fix — Step by Step

Step 1: Recreate the topic with 4 partitions

CLI

```
# Delete old topic (development only!)
kafka-topics.sh --bootstrap-server localhost:9092 --delete --topic orders

# Recreate with 4 partitions
kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --create --topic orders \
  --partitions 4 \
  --replication-factor 1
```

Step 2: Add a key to your producer

Your original producer code was missing the **key** argument. Adding a key makes Kafka distribute messages across partitions based on a hash of the key.

Python — fixed producer with key

```
from confluent_kafka import Producer
from confluent_kafka.admin import AdminClient, NewTopic
import json, time

def delivery_report(err, msg):
    if err is not None:
        print(f'Delivery failed: {err}')
    else:
        print(f'Delivered to {msg.topic()} [partition {msg.partition()}] @ {msg.offset()}')

# Create topic - check the result properly
admin = AdminClient({'bootstrap.servers': 'localhost:9092'})
fs = admin.create_topics([NewTopic('orders', num_partitions=4,
replication_factor=1)])
for topic, f in fs.items():
    try:
        f.result()
        print(f'Topic created: {topic}')
    except Exception as e:
        print(f'Already exists (ok): {e}')
```

```

producer = Producer({'bootstrap.servers': 'localhost:9092'})
i = 0
while True:
    customer_id = f'cust-{i % 10}' # 10 rotating customers
    message = json.dumps({
        'order_id': f'order-{i}',
        'customer_id': customer_id,
        'amount': round(10.0 + (i % 50) * 1.5, 2),
    })
    producer.produce(
        'orders',
        key=customer_id.encode('utf-8'), # <-- KEY ADDED
        value=message.encode('utf-8'),
        on_delivery=delivery_report
    )
    print(f'Produced: order-{i} for {customer_id}')
    i += 1
    producer.poll(0)
    time.sleep(0.5)

```

Expected output after the fix:

Terminal output

```

Produced: order-0 for cust-0
Delivered to orders [partition 2] @ offset 0
Produced: order-1 for cust-1
Delivered to orders [partition 0] @ offset 0
Produced: order-2 for cust-2
Delivered to orders [partition 3] @ offset 0
Produced: order-3 for cust-3
Delivered to orders [partition 1] @ offset 0
# Messages now spread across all 4 partitions

```

■ Second bug in the original code

The original code called `admin_client.create_topics([new_topic])` but never checked the result.

If the topic already existed, the error was silently ignored.

Always iterate over the result and call `f.result()` to detect failures.

Chapter 3

Offset Management — All Commit Types Explained

What an offset is, what `process(msg)` means, and every commit strategy with real code

What Is an Offset?

An offset is a position number on a partition — like a bookmark in a book. Kafka stores your bookmark in a hidden internal topic called `__consumer_offsets`. The committed offset answers one question: *where should I start reading next time?*

If the consumer crashes and restarts, it asks Kafka for the committed offset and resumes from exactly that position. The producer has **nothing to do** with commits — commits are purely the consumer's bookmark.

What Does `process(msg)` Actually Mean?

In all Kafka examples, `process(msg)` is not a Kafka function — it is a placeholder for *your application's work*. A message arrives as raw bytes. You decode it, parse it, and do whatever your system needs. Here are four real examples of what that work looks like.

Example 1 — Save to a database

Python

```
import json, sqlite3

conn = sqlite3.connect('orders.db')
conn.execute('CREATE TABLE IF NOT EXISTS orders (order_id TEXT PRIMARY KEY, amount REAL)')

def process(msg):
    # msg.value() is raw bytes — decode and parse
    data = json.loads(msg.value().decode('utf-8'))
    # YOUR WORK: insert into database
    conn.execute('INSERT OR IGNORE INTO orders VALUES (?,?)',
        [data['order_id'], data['amount']])
    conn.commit()
    print(f"Saved order {data['order_id']} to database")
```

Example 2 — Run an ML model

Python

```
import json
from sklearn.externals import joblib

model = joblib.load('fraud_model.pkl') # load once at startup

def process(msg):
    data = json.loads(msg.value())
    features = [data['amount'], data['hour'], data['country_code']]
    is_fraud = model.predict([features])[0]
    if is_fraud:
        print(f"FRAUD detected: {data['order_id']}")
        send_alert(data)
    else:
        print(f"OK: {data['order_id']}")
```

Example 3 — Call an external API

Python

```
import json, requests

def process(msg):
    data = json.loads(msg.value())
    response = requests.post('https://api.shipping.com/create',
        json={'order_id': data['order_id'],
            'address': data['address']})
    print(f"Shipment created: {response.json()['tracking_id']}")
```

Example 4 — Forward to another Kafka topic

Python

```
import json
from confluent_kafka import Producer

producer = Producer({'bootstrap.servers': 'localhost:9092'})

def process(msg):
    data = json.loads(msg.value())
    enriched = {**data,
        'tax': round(data['amount'] * 0.15, 2),
        'total': round(data['amount'] * 1.15, 2)}
    producer.produce('orders-enriched', value=json.dumps(enriched).encode())
    producer.poll(0)
    print(f"Forwarded {data['order_id']} with tax added")
```

The Three Delivery Guarantees

The timing of your commit relative to your processing work determines what happens when your app crashes. There are three possible guarantees:

Guarantee	Commit timing	Crash risk	Use when
At-most-once	Commit BEFORE processing	Message lost — offset moved, work never done	Metrics, logs
At-least-once	Commit AFTER processing	Message processed twice on restart	Most ML and data pipelines
Exactly-once	Commit INSIDE transaction	Rolls back entirely — no loss, no duplicate	Billing, payments

Commit Type 1 — Auto Commit

Kafka commits your offset in the background every N milliseconds. You do nothing. Risk: if your app crashes between background commits, messages get processed twice on restart.

Python — auto commit

```
consumer = Consumer({
    'bootstrap.servers': 'localhost:9092',
    'group.id': 'auto-group',
    'enable.auto.commit': 'true', # default
    'auto.commit.interval.ms': '5000', # commit every 5 seconds
    'auto.offset.reset': 'earliest',
})
consumer.subscribe(['orders'])

while True:
    msg = consumer.poll(timeout=1.0)
    if msg is None: continue
    if msg.error(): continue
    data = json.loads(msg.value().decode())
    save_to_db(data) # your work
    print(f"Saved {data['order_id']} offset={msg.offset()}")
    # NO commit call – Kafka handles it every 5s silently
```

Terminal output

```
# What happens on crash:
Saved ord-001 | offset=0
Saved ord-002 | offset=1
Saved ord-003 | offset=2
# --- 5 seconds pass, Kafka commits offset 3 ---
Saved ord-004 | offset=3
# --- CRASH here, commit not happened yet ---
# --- restart ---
```

```
Saved ord-004 | offset=3 <-- DUPLICATE! processed again
```

Commit Type 2 — Synchronous Manual Commit

You disable auto-commit and call `commit(asynchronous=False)` yourself after each message is processed. The call blocks — it waits until Kafka confirms the bookmark is saved. Slower but safe.

Python — sync commit

```
consumer = Consumer({
    'bootstrap.servers': 'localhost:9092',
    'group.id': 'sync-group',
    'enable.auto.commit': 'false', # WE control commits
    'auto.offset.reset': 'earliest',
})
consumer.subscribe(['orders'])

while True:
    msg = consumer.poll(timeout=1.0)
    if msg is None: continue
    if msg.error(): continue
    data = json.loads(msg.value().decode())
    # STEP 1: do your work FIRST
    save_to_db(data)
    print(f"Saved {data['order_id']} offset={msg.offset()}")
    # STEP 2: commit AFTER — blocks until broker confirms
    consumer.commit(asynchronous=False)
    print(f"Committed offset={msg.offset()}")
```

Terminal output

```
Saved ord-001 offset=0
Committed offset=0
Saved ord-002 offset=1
Committed offset=1
# --- crash after Saved offset=2, before Committed ---
# --- restart ---
Saved ord-002 offset=2 <-- replayed ONCE (acceptable)
Committed offset=2
```

Commit Type 3 — Asynchronous Commit

Like sync commit but it does not wait. It sends the commit request and immediately moves to the next message. You get notified via a callback if it fails. Best for high-throughput pipelines where a round-trip per message is too slow.

Python — async commit

```

def on_commit(err, partitions):
    if err:
        print(f'COMMIT FAILED: {err}')
    else:
        for p in partitions:
            print(f'Broker confirmed P{p.partition} @ offset {p.offset}')
        consumer = Consumer({
            'bootstrap.servers': 'localhost:9092',
            'group.id': 'async-group',
            'enable.auto.commit': 'false',
            'auto.offset.reset': 'earliest',
            'on_commit': on_commit
        })
        consumer.subscribe(['orders'])
        while True:
            msg = consumer.poll(timeout=1.0)
            if msg is None: continue
            if msg.error(): continue
            save_to_db(json.loads(msg.value().decode()))
            consumer.commit(asynchronous=True) # returns immediately
            print(f'Moved to next, offset={msg.offset()}')

```

Terminal output — notice the interleaving

```

Moved to next, offset=0
Moved to next, offset=1 # already processing next
Broker confirmed P0 @ offset 1 # callback fires later
Moved to next, offset=2

```

Commit Type 4 — Specific Offset Commit (Batch Processing)

Instead of committing 'wherever I am now', you tell Kafka exactly which offset to commit. Most useful when collecting 100 messages, bulk-inserting them into a database in one query, then committing once — saving 99 round-trips.

Python — specific offset / batch commit

```

from confluent_kafka import Consumer, TopicPartition
import json, sqlite3

conn = sqlite3.connect('orders.db')
consumer = Consumer({'bootstrap.servers': 'localhost:9092',
    'group.id': 'batch-group',
    'enable.auto.commit': 'false',
    'auto.offset.reset': 'earliest'})
consumer.subscribe(['orders'])

batch = []

```

```

while True:
    msg = consumer.poll(timeout=1.0)
    if msg and not msg.error():
        batch.append(msg)
        print(f'Collected offset={msg.offset()} (batch={len(batch)})')
        if len(batch) >= 5: # use 100 in production
            rows = [json.loads(m.value().decode()) for m in batch]
            conn.executemany('INSERT OR IGNORE INTO orders VALUES (?,?)',
                [(r['order_id'], r['amount']) for r in rows])
            conn.commit()
            print(f'Bulk inserted {len(batch)} rows')
            # commit highest offset per partition
            offsets = {}
            for m in batch:
                k = (m.topic(), m.partition())
                if k not in offsets or m.offset() > offsets[k].offset - 1:
                    offsets[k] = TopicPartition(m.topic(), m.partition(),
                        m.offset() + 1) # +1 = next to read
            consumer.commit(offsets=list(offsets.values()), asynchronous=False)
            print(f'Committed: {[ (k, v.offset) for k, v in offsets.items() ]}')
            batch = []

```

Terminal output

```

Collected offset=0 (batch=1)
Collected offset=1 (batch=2)
Collected offset=2 (batch=3)
Collected offset=3 (batch=4)
Collected offset=4 (batch=5)
Bulk inserted 5 rows
Committed: [ (('orders', 0), 5)] # one commit for all 5

```

✓ Recommended pattern for most applications

Disable auto commit: `enable.auto.commit = false`

Process the message completely first (write to DB, run model, call API)

Then commit the offset — this gives at-least-once delivery

Make your processing idempotent (safe to run twice) using INSERT OR IGNORE or similar

Chapter 4

Auto-Commit Deep Dive — Three Real Scenarios

Answers to specific questions about when commits happen relative to producers and consumers

One Critical Point Before the Scenarios

■ The commit has nothing to do with the producer

The producer just writes messages to Kafka's log.

The commit is purely the consumer's bookmark — it only exists when a consumer is running.

Kafka stores messages the moment the producer sends them, whether or not any consumer exists.

The committed offset only moves when a consumer reads and commits.

YOUR QUESTION

Q1: Producer runs and delivers messages 0–500. Consumer is OFF the whole time. Does Kafka write any commits?

No. Kafka commits nothing while the consumer is off. The messages are stored on disk the moment the producer sends them — but the bookmark does not move because there is no consumer to move it.

When the consumer starts with `auto.offset.reset = 'earliest'`, Kafka checks: 'does this group have a committed offset?' — finds none — and starts from offset 0. The consumer reads all 501 messages that piled up, and the auto-commit timer begins from that point.

Python

```
# Producer ran, delivered offsets 0-500. Consumer was off.
# Consumer starts now:
consumer = Consumer({
    'bootstrap.servers': 'localhost:9092',
    'group.id': 'my-group',
    'enable.auto.commit': 'true',
    'auto.commit.interval.ms': '5000',
```

```
'auto.offset.reset': 'earliest', # no bookmark -> start from 0
})
consumer.subscribe(['orders'])

# What happens:
# Kafka: no committed offset for 'my-group' -> start from 0
# consumer reads: offset 0, 1, 2 ... 500
# at 5 second mark: auto-commit fires, committed offset = wherever we are
```

YOUR QUESTION

Q2: Producer sends 6 messages per second (10ms each). Commit interval = 3000ms. Does it commit for each message or for all messages in that 3 seconds?

It commits ALL messages read in that 3-second window in **one single operation**. Auto-commit does not commit one message at a time. It commits 'wherever I am right now' every 3 seconds. After 3 seconds of reading 6 messages/second, 18 messages have been read — and offset 18 is committed in a single step.

Timeline

```
# 6 messages/sec, commit every 3s
# time event
# 0.01s Read offset 0 -> Order 1
# 0.02s Read offset 1 -> Order 2
# ... (reading continuously)
# 1.00s Read offset 5 -> Order 6 (1s = 6 messages)
# 2.00s Read offset 11 -> Order 12 (2s = 12 messages)
#
# 3.00s AUTO-COMMIT FIRES
# Commits offset 18 — ONE commit for all 18 messages
#
# 3.01s Read offset 18 -> Order 19
# 6.00s AUTO-COMMIT FIRES AGAIN — commits offset 36
#
# --- CRASH at 5.5s ---
# Last commit was at 3s (offset 18)
# Messages 19-33 were processed but NOT committed
# Restart: replays from offset 18 -- 15 messages reprocessed
```

YOUR QUESTION

Q3: Producer sends 1 message every 2 seconds. Commit interval = 1 second. What happens?

The commit timer fires every 1 second, but it only commits if something new was read since the last commit. If no messages arrived in that 1-second window, nothing is written. In this scenario, each message gets its own commit — but that is a coincidence of timing, not a per-message design.

Timeline

```
# 1 message every 2s, commit every 1s
# time event
# 1.0s commit timer fires -- nothing read yet, no commit
# 2.0s Read offset 0 -> Order 1
# 3.0s commit timer fires -- new message! committed offset = 1
# 4.0s Read offset 1 -> Order 2
# 5.0s commit timer fires -- new message! committed offset = 2
# 6.0s Read offset 2 -> Order 3
# 7.0s commit timer fires -- committed offset = 3
#
# Each message is committed ~1 second after it is read.
# The commit timer is smart: it does nothing when there is nothing new.
```

Chapter 5

Independent Consumer Groups — How Offsets Are Isolated

Why Consumer B starts from offset 0 even though Consumer A already read to offset 100

The Question That Led Here

YOUR QUESTION

Topic 'orders' has messages 0–500. Consumer A (group-A) reads 0–100 then stops. Now Consumer B starts with a DIFFERENT group.id (group-B). Does B start from 0 or 100?

Consumer B starts from **offset 0**. The committed offset is stored per group.id — not per topic. Group-B has never connected before so Kafka has no bookmark for it, and with `auto.offset.reset='earliest'` it starts from the beginning.

How Kafka Stores Offsets

Kafka's internal `__consumer_offsets` topic stores entries keyed by (group.id, topic, partition). Each group has a completely independent bookmark:

Internal state

```
# What Kafka stores internally:
(group-A, orders, partition-0) -> offset 101 # A stopped here
(group-B, orders, partition-0) -> NO ENTRY # B never connected

# When group-B consumer starts:
# Kafka checks: any entry for (group-B, orders, 0)? -> No
# auto.offset.reset = 'earliest' -> start from offset 0
```

Proof in Code

Python — Consumer A

```
# Consumer A — reads 0->100 then stops
consumer_a = Consumer({
```

```
'bootstrap.servers': 'localhost:9092',
'group.id': 'group-A',
'enable.auto.commit': 'true',
'auto.offset.reset': 'earliest',
})
consumer_a.subscribe(['orders'])
count = 0
while count < 100:
    msg = consumer_a.poll(1.0)
    if msg and not msg.error():
        print(f'[A] offset={msg.offset()}')
        count += 1
consumer_a.close() # commits final offset on clean shutdown
print('Consumer A stopped. group-A bookmark = 101')
```

Python — Consumer B

```
# Consumer B – completely new group
consumer_b = Consumer({
'bootstrap.servers': 'localhost:9092',
'group.id': 'group-B', # different group.id
'enable.auto.commit': 'true',
'auto.offset.reset': 'earliest', # no bookmark -> start from 0
})
consumer_b.subscribe(['orders'])
while True:
    msg = consumer_b.poll(1.0)
    if msg and not msg.error():
        print(f'[B] offset={msg.offset()}')
```

Terminal output

```
[B] offset=0 value=Order 0 <- starts from 0, not 100!
[B] offset=1 value=Order 1
[B] offset=2 value=Order 2
...
[B] offset=99 value=Order 99 <- reads what A already read
[B] offset=100 value=Order 100
...
[B] offset=500 value=Order 500 <- reads everything up to 500
```

Verify with CLI

CLI

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --list
# group-A
# group-B
```

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \  
--group group-A --describe  
# TOPIC PART CURRENT-OFFSET LAG  
# orders 0 101 400 <- A is at 101, still 400 behind  
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \  
--group group-B --describe  
# TOPIC PART CURRENT-OFFSET LAG  
# orders 0 501 0 <- B read everything
```

The Real-World Power of This Design

Multiple completely different services can all read the same topic, each at a different position, without interfering with each other:

Conceptual

```
# All three read the SAME 'orders' topic  
# Each has its own independent bookmark  
group.id: 'fraud-detection' # at offset 500, real-time  
group.id: 'email-service' # at offset 498, slightly behind  
group.id: 'analytics' # at offset 450, processing slower  
# None of them interfere with each other.  
# If analytics crashes, fraud-detection keeps running without any impact.
```

Chapter 6

Exactly-Once Transactions — Full Code Walkthrough

The bank transfer analogy, every config variable explained, and complete working code

The Analogy — Bank Transfer

Imagine a bank transfer: you read the instruction 'transfer £100 from A to B', you debit account A, and you credit account B. If the system crashes after debiting A but before crediting B — £100 has vanished. You need all three steps to happen together or not at all. That is **atomicity**.

In Kafka, the three steps are: **read the input message**, **write the output message**, and **move the consumer bookmark**. Without a transaction, a crash between steps 2 and 3 means the output message is visible to downstream consumers but the offset never moved — so on restart you process the same input again and produce a duplicate. A Kafka transaction wraps all three into one atomic unit.

Step	Without transaction	With transaction
1. Read msg	normal poll()	normal poll()
2. Produce output	visible immediately	PENDING — hidden
3. Commit offset	separate call	inside transaction
Crash between 2&3	duplicate output!	rolls back, retry clean

Every Variable and Function Explained

`enable.idempotence = true`

What it does: Prevents duplicate messages when the network hiccups. Without it, when a network error causes the producer to retry, Kafka receives the same message twice and writes it twice. With idempotence, Kafka gives the producer a unique Producer ID (PID) and a sequence number per message. If the same PID+sequence arrives again, Kafka silently discards the duplicate.

Analogy: Like a cashier who remembers your order number — if you say 'I ordered pizza #42' twice, they serve it only once.

Python

```
# Without idempotence:
# Producer sends 'Order 5' -> network drops the acknowledgement
# Producer thinks: no ack, I must retry
# Kafka already has 'Order 5' at offset 10
# Retry arrives -> Kafka writes 'Order 5' AGAIN at offset 11 = DUPLICATE

# With idempotence:
producer = Producer({
    'bootstrap.servers': 'localhost:9092',
    'enable.idempotence': 'true',
    # auto-set when idempotence is true:
    # acks = 'all' (wait for ALL replicas)
    # retries = INT_MAX (retry forever)
    # max.in.flight = 5 (keep ordering)
})
# message envelope: {pid: 7, seq: 42, value: 'Order 5'}
# Retry arrives -> Kafka sees pid=7 seq=42 again -> discard
# offset 10 -> Order 5 only ONE copy
```

isolation.level = read_committed

What it does: Makes the consumer ignore messages that are still inside an open transaction. It only sees messages after `commit_transaction()` fired. The default is **read_uncommitted** which sees pending messages that might get rolled back.

Analogy: Like a restaurant menu that only shows dishes that are actually ready to serve — not dishes still being cooked.

Python

```
# Timeline:
# 0.1s producer.produce('orders-confirmed', 'Order 5')
# -> written to log, marked PENDING
#
# read_uncommitted consumer: sees 'Order 5' RIGHT NOW
# read_committed consumer: sees NOTHING (waiting)
#
# 0.2s producer CRASHES – transaction never committed
# Kafka rolls back – 'Order 5' deleted
#
# read_uncommitted: already processed it -> PROBLEM
# read_committed: saw nothing -> safe
consumer = Consumer({
    'bootstrap.servers': 'localhost:9092',
```

```
'group.id': 'safe-group',
'isolation.level': 'read_committed', # only see committed messages
})
```

producer.begin_transaction()

What it does: Opens an invisible envelope. Everything produced after this call goes into the envelope and is hidden from read_committed consumers until you call commit_transaction(). If you call abort_transaction(), the envelope is thrown away and all pending messages disappear.

Analogy: Sealing items in a box before shipping. Nothing is delivered until the box is sealed and sent. If you cancel, everything in the box is discarded.

Python

```
producer = Producer({
    'bootstrap.servers': 'localhost:9092',
    'transactional.id': 'order-processor-1', # unique per producer instance
    'enable.idempotence': 'true',
})
producer.init_transactions() # registers with broker, gets epoch
# epoch prevents old zombie producers interfering
producer.begin_transaction() # open the envelope
producer.produce('orders-confirmed', value=b'Order 5') # PENDING
producer.produce('audit-log', value=b'logged') # also PENDING
# both messages in the envelope, not visible yet
```

■ transactional.id must be unique per worker

If you run 3 workers: processor-1, processor-2, processor-3

If two workers share the same transactional.id, Kafka fences out the older one (raises error).

This prevents split-brain where two workers process the same message simultaneously.

producer.send_offsets_to_transaction(...)

What it does: Puts the consumer's offset commit INSIDE the transaction envelope. Instead of committing the offset in a separate call (which could fail independently), it is now part of the same atomic operation. Either the offset moves AND the output appears, or neither happens.

Python — with explanation

```
# msg is at: topic='orders', partition=0, offset=42
producer.send_offsets_to_transaction(
    # Argument 1: which offset to commit
    # offset + 1 means 'next time start from the message AFTER this one'
```

```

{msg.topic(): {msg.partition(): msg.offset() + 1}},
# concrete: {'orders': {0: 43}}
# Argument 2: which consumer group owns this offset
# Kafka needs group.id + generation to know where to store the bookmark
consumer.consumer_group_metadata()
)
# This is INSIDE the envelope – only applied when commit_transaction() fires
# If abort: offset stays at 42, same message retried

```

commit_transaction() and abort_transaction()

commit_transaction(): Seals and delivers the envelope. At this single instant: the produced output becomes visible to downstream consumers, the consumer offset is moved forward, and the transaction is marked complete. All three happen simultaneously.

abort_transaction(): Throws the envelope away. Called in the except block when something goes wrong. The pending output message disappears. The offset stays where it was. The next loop iteration retries the same message from the same offset. Zero duplicates.

The Complete Annotated Code

Python — complete annotated EOS code

```

from confluent_kafka import Consumer, Producer
import json

# CONSUMER: reads from input topic
consumer = Consumer({
    'bootstrap.servers': 'localhost:9092',
    'group.id': 'eos-group',
    'enable.auto.commit': 'false', # offset committed INSIDE transaction
    'isolation.level': 'read_committed', # only see committed transaction output
    'auto.offset.reset': 'earliest',
})
consumer.subscribe(['orders'])

# PRODUCER: writes to output topic
producer = Producer({
    'bootstrap.servers': 'localhost:9092',
    'transactional.id': 'order-processor-1', # unique per instance
    'enable.idempotence': 'true',
})
producer.init_transactions() # must call before begin_transaction()

while True:
    msg = consumer.poll(timeout=1.0)
    if msg is None: continue
    if msg.error(): continue
    data = json.loads(msg.value().decode())

```

```

print(f"Read: {data['order_id']} offset={msg.offset()}")
try:
    # OPEN ENVELOPE
    producer.begin_transaction()
    # YOUR WORK: transform the data
    enriched = {
        **data,
        'tax': round(data['amount'] * 0.15, 2),
        'total': round(data['amount'] * 1.15, 2),
        'status': 'confirmed'
    }
    # INSIDE ENVELOPE: write to output topic (PENDING, not visible yet)
    producer.produce(
        'orders-confirmed',
        key=data['order_id'].encode(),
        value=json.dumps(enriched).encode()
    )
    print('Produced enriched order (pending, not visible yet)')
    # INSIDE ENVELOPE: include consumer offset commit
    producer.send_offsets_to_transaction(
        {msg.topic(): {msg.partition(): msg.offset() + 1}},
        consumer.consumer_group_metadata()
    )
    # SEAL AND DELIVER: output visible + offset moved simultaneously
    producer.commit_transaction()
    print('Transaction committed – output visible, offset moved')
except Exception as e:
    print(f'Error: {e} – aborting')
    producer.abort_transaction()
    # pending output deleted, offset unchanged, same msg retried next loop

```

Terminal Output — Both Paths

Terminal output

```

# Happy path:
Read: ord-001 offset=0
Produced enriched order (pending, not visible yet)
Transaction committed – output visible, offset moved
Read: ord-002 offset=1
Produced enriched order (pending, not visible yet)
Transaction committed – output visible, offset moved

# Crash path:
Read: ord-003 offset=2
Produced enriched order (pending, not visible yet)
# --- CRASH before commit_transaction() ---

```

```
# --- Kafka rolls back: pending message deleted, offset stays at 2 ---  
# --- restart ---  
Read: ord-003 offset=2 <- retried cleanly, no duplicate in output  
Produced enriched order (pending, not visible yet)  
Transaction committed – output visible, offset moved
```

✓ When to use exactly-once

Use EOS for: billing, payment processing, order fulfilment — anywhere duplicates cause real damage.

For ML feature pipelines and analytics: at-least-once + idempotent processing is usually sufficient.

EOS adds ~10-20% latency overhead due to the two-phase commit protocol.

Always pair with `isolation.level=read_committed` on all downstream consumers.